



FOR JAVA DEVELOPERS MOVING TO NESTJS

Filter $\neq$ Middleware

Spring Boot to NestJS, episode 9: middleware and interceptors

`chain.doFilter` hands the error back. `next()` does not. The interceptor wraps the handler.

A Spring filter catches downstream errors because the exception travels back up through `chain.doFilter`. NestJS middleware does not get the controller exception back through `next()`.

Measured: the handler runs inside `next()` and throws, `next()` returns normally, and the 500 is written afterwards. The interceptor is the hook that wraps the handler, so it sees both the result and the error.

Versions on this sheet were current on 30 September 2026: Node 22, NestJS 12, Spring Boot 4.1.1.

In scope

- Why a try/catch around `next()` catches nothing, measured
- Where each NestJS hook runs on one request
- Reshaping every response with one interceptor
- Why `HandlerInterceptor` is the wrong Spring match, and `ResponseBodyAdvice` the right one

Not in scope

- Guards and authorization: a later episode
- Express error-handling middleware: for controller failures, stay with Nest filters and interceptors

No single Spring class maps to a Nest interceptor. Map capabilities, not names.

01 The measurement

SPRING FILTER	<code>filter CAUGHT ServletException (wrapping IllegalStateException)</code>
NEST MIDDLEWARE	<code>caught nothing; saw status 500 when the response finished</code>
NEST INTERCEPTOR	<code>CAUGHT the database refused the write</code>
<code>next()</code> AROUND A THROW	<code>handler threw inside next(); next() returned; headersSent=false</code>
ONE <code>map()</code>	<code>[...] becomes {"data":[...], "meta":{"path":"GET /payments", "tookMs":0}}</code>
<code>postHandle</code>	<code>ran after ResponseBodyAdvice; handed ModelAndView=null; skipped on the throw</code>

The same handler throwing on both stacks, then reshaping a response.

02 The interceptor that sees the error

An interceptor receives the handler result as a stream from `next.handle()`, so both the return value and the thrown error flow through it.

```
nestjs-api/src/ep08-middleware/exception-visibility.ts

class CatchingInterceptor implements NestInterceptor {
  intercept(_ctx: ExecutionContext, next: CallHandler): Observable<unknown> {
    return next.handle().pipe(
      catchError((e) => {
        seen.push(`interceptor CAUGHT ${e as Error}.message`);
        return throwError(() => e);
      }),
    );
  }
}
```

Interceptors can transform the result returned from a function and the exception thrown from it

NestJS documentation, Request lifecycle and Interceptors, checked 2026-09-30.

03 One map(), every response

The handler does not change. Every response that goes through this interceptor gets the same envelope.

```
nestjs-api/src/ep08-middleware/transform.ts

return next.handle().pipe(
  map((data) => ({
    data,
    meta: {
      path: `${method} ${url}`,
      tookMs: Math.round(Number(process.hrtime.bigint() - started) / 1e6),
    },
  })),
);
```

handler returns `[{"id":"pay_1","amountInMinorUnits":4500}]`; client receives `{"data":[...],"meta":{...}}`

Run on a real machine, 2026-09-30, by `scripts/verify-middleware.sh` in the course repository.

04 The Spring answer: `ResponseBodyAdvice`

`HandlerInterceptor.postHandle` is the name match and the wrong job for a REST body: the advice ran first, and `postHandle` got no model. In this app a header set in `postHandle` still reached the client because the small response was buffered; Spring documents `postHandle` as too late for `@ResponseBody`, so do not rely on it.

Ep08BodyRewriteTest.java

```
@RestControllerAdvice
static class EnvelopeAdvice implements ResponseBodyAdvice<Object> {
    @Override
    public Object beforeBodyWrite(Object body, MethodParameter returnType, MediaType
                                contentType,
                                Class<?> extends HttpMessageConverter<?>> converterType,
                                ServerHttpRequest request, ServerHttpResponse response) {
        var envelope = new LinkedHashMap<String, Object>();
        envelope.put("data", body);
        envelope.put("meta", Map.of("path", "GET " + request.getURI().getPath()));
        return envelope;
    }
}
```

For `@ResponseBody` and `ResponseEntity` methods the response is written within the `HandlerAdapter` before `postHandle`; use `ResponseBodyAdvice`

Spring Framework reference, Interception, checked 2026-09-30.

Which hook

1. The raw request, before anything else: middleware
2. Who may call this route: a guard
3. Check or convert an argument: a pipe
4. See or reshape the result or the error: an interceptor
5. Turn an exception into a response: an exception filter
6. Never catch downstream errors in middleware



Everything else

- | | |
|--|---|
|  Site
https://code-with-sam-dev.github.io |  Facebook
https://www.facebook.com/codewithsamdev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  GitHub
https://github.com/code-with-sam-dev |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  x
https://x.com/CodeWithSamDev |  Buy me sadza
https://buymesadza.com/codewithsam |
|  TikTok
https://www.tiktok.com/@codewithsamdev | Every measurement, one command: scripts/verify-middleware.sh
https://github.com/code-with-sam-dev/spring-to-node |
|  Instagram
https://www.instagram.com/codewithsamdev | NestJS: Interceptors
https://docs.nestjs.com/interceptors |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.